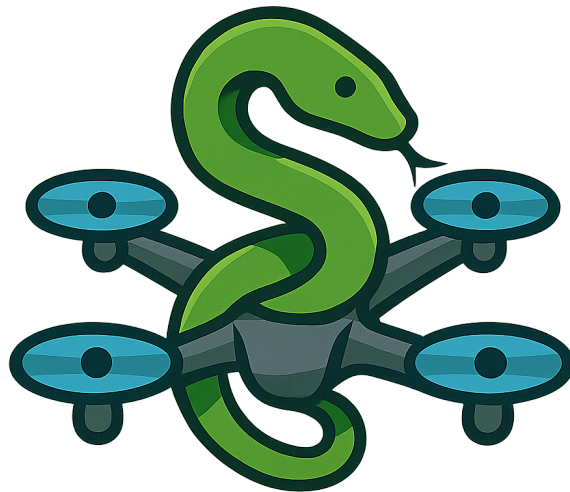


Technische Hochschule Ostwestfalen-Lippe

Bachelor of Science, Data Science

Gamification



Ausarbeitung zum Thema:

Vom Prototyp zur Spielschleife

Gamifizierung von *PythonWithDrones*

Erstellt von:	Niclas Falke, Stefan Reitemeyer und Jonas Pottmeier
Matrikelnummer:	20122542 (Niclas), 20124913 (Stefan) und 20116026 (Jonas)
Fachsemester:	4
Abgabedatum:	28.09.2026
Prüfer:	Prof. Dr.-Ing. Rainer Rasche

Inhaltsverzeichnis

Abstrakt	1
1 Einleitung	1
2 Stand der Forschung	1
3 Ausgangslage	2
3.1 Der Stand zum Modulwechsel	2
3.2 Grenzen des Ausgangsstands	2
4 Begriffe und Festlegungen	3
4.1 Gamifizierung	3
4.2 Mechanik, Dynamik und Erfahrung	3
4.3 Drei Grundbedürfnisse	3
4.4 Passung von Anforderung und Fähigkeit	3
4.5 Festlegungen für jedes Spielelement	4
5 Methodik	4
5.1 Überblick	4
5.2 Erweiterter Handlungsraum der Drohne	4
5.2.1 Neue Befehle	4
5.2.2 Lösungsbedingungen	5
5.3 Aufgabenkurve	5
5.4 Kennzahlen und Rückmeldung eines Laufs	5
5.4.1 Kennzahlen	5
5.4.2 Terminal	5
5.5 Architektur der Bestenliste	6
5.5.1 Was gewertet wird	6
5.5.2 Betrieb ohne Backend	6
5.6 Level-Editor	7
5.6.1 Aufbau	7
5.6.2 Vom Entwurf zum Level	7
5.7 Reibungsabbau	7
6 Ergebnisse	8
6.1 Neue Bausteine und Levelübersicht	8
6.2 Rückmeldung und Bestenliste	9
6.3 Level-Editor und mobile Darstellung	12
7 Diskussion	14
7.1 Zuordnung zu den Grundbedürfnissen	14
7.2 Einschränkungen und Reichweite der Aussagen	14
7.3 Bewusst getroffene Festlegungen	14
7.4 Wertungslogik und Motivation	15
8 Schlussfolgerung	15
8.1 Zusammenfassung	15
8.2 Nächste Schritte	15
Literatur	16

Abstrakt

PythonWithDrones ist eine Lernanwendung, in der Python-Quelltext eine virtuelle Drohne durch dreidimensionale Level steuert. Zum Abschluss des Vorgängermoduls lag ein lauffähiger Prototyp vor, dessen Ablauf mit dem Erreichen des Zielportals endete [1]. Danach war die Interaktion beendet. Die vorliegende Arbeit beschreibt, wie diese Anwendung zwischen dem 10. April und dem 28. September 2026 zu einem Spiel mit geschlossener Schleife ausgebaut wurde. Hinzugekommen sind ein erweiterter Handlungsraum der Drohne mit Schieben, Aufnehmen, Abliefern und Vorausschauen, eine auf dreizehn Level verlängerte Aufgabenkurve, eine Auswertung jedes Laufs nach Abstürzen, geflogenen Feldern und Codezeilen, eine serverseitige Bestenliste mit Kontoverwaltung und Versuchsverlauf sowie ein dreidimensionaler Level-Editor, mit dem Spielende eigene Aufgaben bauen und einreichen können. Begleitet wird das von einer überarbeiteten mobilen Darstellung, einer eigenen Fehlerseite und einer automatisierten Testabdeckung. Die Auswertung ordnet die umgesetzten Elemente den drei Grundbedürfnissen Kompetenz, Autonomie und soziale Eingebundenheit zu und begründet, wie die einzelnen Entwurfsentscheidungen zusammenwirken.

1 Einleitung

Zum Abschluss des vorangegangenen Moduls lag eine funktionsfähige Anwendung vor. Python wurde im Browser ausgeführt, die Drohne ließ sich steuern, und die vorhandenen Level waren lösbar. Die Anwendung unterschied je Level jedoch nur zwei Zustände, das Erreichen oder das Nichterreichen des Zielportals. Der Umfang einer Lösung, die Zahl der Fehlversuche und die Frage, ob nach Abschluss des letzten Levels ein Anlass zur weiteren Nutzung bestand, blieben unberücksichtigt.

Die vorliegende Arbeit hat das Ziel, diese Anwendung um Spielelemente zu erweitern und die dabei getroffenen Entwurfsentscheidungen zu begründen. Untersucht wird, welche Elemente aus dem Spieldesign sich in eine bestehende Lernanwendung einfügen lassen, welche Wirkung strukturell von ihnen zu erwarten ist und welche Kosten sie im Betrieb verursachen. Die Ausführung von Python im Browser ist nicht Gegenstand der Arbeit, da sie im Vorgängermodul entwickelt wurde [1]. Gamifizierung wird nicht als nachträgliche Ergänzung einer fertigen Anwendung verstanden, sondern als Eingriff in deren Regeln, der verändert, worauf Lernende beim Schreiben ihres Programms achten.

Die Arbeit stützt sich auf den Quelltext des Projekts und seine Versionsgeschichte [2] sowie auf die Ausarbeitung des Vorgängermoduls [1]. Als Ausgangspunkt dient das Tag v1.0.0 vom 10. April 2026, das den Stand zum Abschluss des Vorgängermoduls markiert. Der Betrachtungszeitraum umfasst die Änderungen zwischen diesem Stand und dem 28. September 2026.

2 Stand der Forschung

Die Umsetzung baut auf einem etablierten Satz an Werkzeugen aus dem Web- und Backend-Bereich auf, ohne selbst neue Technologie einzuführen. Next.js dient als React-Framework für den statischen Seitenexport [3]. Pyodide bringt einen zu WebAssembly

übersetzten CPython-Interpreter in den Browser und macht damit eine serverseitige Python-Ausführung überflüssig [4]. React Three Fiber bindet die 3D-Bibliothek Three.js deklarativ in React-Komponenten ein und trägt die im Browser laufende Drohnenszene [5]. Auf der Serverseite stellt FastAPI die asynchrone Web-Schnittstelle der Bestenliste bereit [6], SQLAlchemy vermittelt den objektrelationalen Zugriff darauf [7], und PostgreSQL hält die Daten persistent [8]. Alle fünf Bausteine sind quelloffen, gut dokumentiert und bereits im Vorgängermodul beziehungsweise im bestehenden Projekt im Einsatz [1]. Die vorliegende Arbeit erweitert diesen Stack um die in Abschnitt 3 beschriebenen Fähigkeiten, statt ihn auszutauschen.

3 Ausgangslage

3.1 Der Stand zum Modulwechsel

Die Anwendung ist ein statischer Export eines Next.js-Projekts [3] und wird über GitHub Pages ausgeliefert. Sie benötigte zu diesem Zeitpunkt keine serverseitige Komponente. Geschriebener Quelltext wird an einen Web Worker übergeben, in dem Pyodide einen zu WebAssembly übersetzten CPython-Interpreter betreibt [4]. Jeder Drohnenbefehl erzeugt dort ein Datenobjekt, das über `postMessage` an den Hauptthread zurückgereicht wird. Dort nimmt eine Warteschlange die Objekte entgegen und arbeitet sie nacheinander als Animationen in einer mit React Three Fiber aufgebauten Szene ab [5]. Der Fortschritt liegt im `localStorage` des Browsers. Der Aufbau dieses Prototyps ist in der Ausarbeitung des Vorgängermoduls ausführlich beschrieben [1].

Level sind YAML-Dateien. Sie beschreiben den Startpunkt der Drohne und eine Folge horizontaler Schichten, in denen jede Zelle über eine Block-Kennung belegt ist. Welche Eigenschaften eine solche Kennung hat, ob ein Block also eine Kollision auslöst oder als Ziel gilt, steht zentral in der Blockregistrierung. Sowohl die Python-Seite als auch die Darstellung greifen auf dieselbe Quelle zu.

Zum Zeitpunkt von `v1.0.0` umfasste dieser Aufbau sechs spielbare Level, neun Blocktypen und zehn öffentliche Methoden am Drohnenobjekt. Der Handlungsraum bestand aus Bewegen, Steigen, Sinken, Drehen und zwei Abfragen, mit denen sich ein blockierter Weg und das Erreichen des Portals prüfen ließen.

3.2 Grenzen des Ausgangsstands

Aus der Perspektive des Gamification-Moduls fehlten dem beschriebenen Aufbau drei Voraussetzungen. Zunächst gab es keine Bewertung einer Lösung. Die Anwendung erfasste allein deren Vorhandensein und bot damit keine Grundlage für eine Rückmeldung, die über die Feststellung des Erfolgs hinausgeht, und keinen Maßstab für Verbesserung. Weiter gab es keinen Bezug zu anderen Nutzenden, da der Fortschritt ausschließlich lokal im Browser abgelegt war. Zwei Personen, die dasselbe Level lösten, erfuhren nichts voneinander, und ein Gerätewechsel verwarf den Fortschritt. Schließlich gab es keinen Spielraum für eigene Inhalte. Level lagen als YAML-Dateien im Repository, und ein eigener Beitrag setzte Kenntnis der Dateistruktur, ein Auschecken des Projekts und einen Pull Request voraus, was für die Zielgruppe keine erreichbare Handlung darstellt.

4 Begriffe und Festlegungen

Dieser Abschnitt legt fest, in welcher Bedeutung die zentralen Begriffe im weiteren Verlauf verwendet werden und woran die Entwurfsentscheidungen anschließend gemessen werden.

4.1 Gamifizierung

Gamifizierung bezeichnet in dieser Arbeit den Einsatz einzelner Elemente aus dem Spieldesign in einem Zusammenhang, der selbst kein Spiel ist [9]. Maßgeblich ist die Beschränkung auf einzelne Elemente. Die Lernanwendung wird nicht zu einem Spiel umgebaut. Übernommen werden stattdessen einzelne Bausteine, deren Wirkung aus Spielen bekannt ist. Die drei am häufigsten übernommenen Bausteine sind Punkte, Abzeichen und Bestenlisten [10]. Punkte machen eine Leistung zählbar, Abzeichen machen sie sichtbar, und Bestenlisten machen sie vergleichbar. Im vorliegenden Projekt sind die Punkte als Kennzahlen je Lauf und die Bestenliste umgesetzt, während Abzeichen zurückgestellt sind.

4.2 Mechanik, Dynamik und Erfahrung

Der Entwurf trennt drei Ebenen, wie sie im MDA-Rahmenwerk des Spieldesigns beschrieben sind [11]. Die Mechanik ist die Regel, die im Quelltext geschrieben wird. Die Dynamik ist das Verhalten, das im Betrieb daraus entsteht. Die Erfahrung ist das Ergebnis, das bei den Spielenden ankommt. Die Trennung ist nützlich, weil auf der Ebene der Mechanik gebaut wird, während die Wirkung auf der Ebene der Erfahrung eintritt und die dazwischenliegende Dynamik sich nur mittelbar festlegen lässt.

Im vorliegenden Projekt lässt sich das an der Bestenliste zeigen. Die Mechanik ist deren Sortierreihenfolge. Die daraus entstehende Dynamik ist, dass Spielende ihren Lösungsweg kürzen, statt ihn nur lauffähig zu machen. Die Erfahrung ist die eines gefundenen, kompakten Lösungswegs. Die Aufgabe des Entwurfs besteht darin, die Mechanik so zu wählen, dass die beabsichtigte Dynamik entstehen kann.

4.3 Drei Grundbedürfnisse

Als Ordnungsrahmen dienen drei Bedürfnisse aus der Selbstbestimmungstheorie, die eine Tätigkeit aus sich heraus tragen können [12]. Kompetenz bezeichnet das Erleben, besser zu werden. Das setzt voraus, dass Fortschritt abgestuft sichtbar ist und ein Fehlschlag eine benennbare Ursache hat. Autonomie bezeichnet den Spielraum, eigene Entscheidungen zu treffen, bis hin zur Wahl und zum Bau einer eigenen Aufgabe. Soziale Eingebundenheit bezeichnet den Bezug zu anderen, sei es über einen Vergleich, einen sichtbaren Namen oder geteilte Inhalte. Das Raster macht die Verteilung der umgesetzten Elemente sichtbar und zeigt, ob ein Entwurf alle drei Bedürfnisse adressiert.

4.4 Passung von Anforderung und Fähigkeit

Eine Aufgabe trägt dann, wenn ihre Anforderung und die Fähigkeit der Lösenden in einem engen Verhältnis stehen, das der Flow-Theorie als Flow-Kanal beschrieben wird [13]. Liegt die Anforderung darüber, entsteht Überforderung. Liegt sie darunter,

entsteht Langeweile. Für eine Lernanwendung mit fester Reihenfolge folgt daraus, dass die Steigerung zwischen zwei Leveln weder zu klein noch zu groß sein darf und dass erkennbar sein sollte, was ein Level verlangt, bevor es geöffnet wird.

4.5 Festlegungen für jedes Spielelement

Aus dem Zweck der Anwendung ergeben sich zwei Festlegungen, die für jedes eingebaute Spielelement gelten. Die gemessenen Größen sind Eigenschaften der Lösung und nicht des Verhaltens, das zu ihr geführt hat. Eine Wertung, die belohnt, wer am meisten Zeit aufwendet oder am schnellsten tippt, erfasst nicht den vermittelten Inhalt, während eine Wertung nach der Zahl der Schritte das Verständnis des Levels erfasst. Zudem steuert keine Belohnung den Zugang zu Lerninhalten. Jedes Level bleibt ohne Konto, ohne Bestenliste und ohne Serverbetrieb vollständig spielbar. Ein Spielelement darf den Reiz einer Aufgabe erhöhen, ihn aber nicht ersetzen, und es darf niemanden ausschließen.

5 Methodik

5.1 Überblick

Die Änderungen im Betrachtungszeitraum lassen sich in sechs Arbeitssträngen zusammenfassen, die aufeinander aufbauen. Der erweiterte Handlungsraum ist Voraussetzung für Level mit mehr als einem Lösungsweg, die erhobenen Kennzahlen sind Voraussetzung für eine sortierbare Bestenliste. Der Handlungsraum der Drohne wuchs um Schieben, Aufnehmen, Abliefern und Vorausschauen, dazu kamen fünf neue Blocktypen und Lösungsbedingungen im Levelformat. Darauf aufbauend wuchs die Aufgabenkurve von sechs auf dreizehn Level bei überarbeiteter Einstiegsreihe und Schlagworten als Vorschau. Die Laufauswertung erfasst seither Abstürze, geflogene Felder und Codezeilen je Lauf und zeigt sie nach Abschluss an. Ein Terminal gibt zusätzlich `print`-Ausgaben aus. Auf dieser Grundlage entstand eine Bestenliste als FastAPI-Dienst mit Postgres-Datenbank, Konten, JWT-Anmeldung, Rangfolge und Versuchsverlauf. Parallel dazu wurde ein dreidimensionaler Level-Editor mit Testlauf, YAML-Export und Einreichung über GitHub gebaut. Der sechste Strang bündelt Reibungsabbau, darunter die Darstellung auf schmalen Bildschirmen, eine eigene Fehlerseite und eine automatisierte Testabdeckung.

5.2 Erweiterter Handlungsraum der Drohne

5.2.1 Neue Befehle

Solange die Drohne nur fliegen und sich drehen konnte, bestand jedes Ziel darin, eine Position zu erreichen. Level unterschieden sich allein in der Geometrie des Weges. Vier neue Befehle verändern oder prüfen seither den Zustand der Welt. `drone.push()` schiebt einen schiebbaren Block ein Feld weiter und rückt selbst nach. Ist das Zielfeld belegt, bleibt die Aktion folgenlos und nennt den Grund. `drone.pickup()` und `drone.deliver()` nehmen ein Paket auf und legen es wieder ab. Beide melden, wenn nichts aufzunehmen oder abzulegen ist. `drone.scan(n)` liefert die Block-Kennungen der nächsten `n` Felder in

Blickrichtung und bricht am ersten blockierenden Feld ab, ohne Argument nur die eine nächste Kennung.

`scan` ersetzt die frühere Abfrage `is_path_blocked`, die nur einen Wahrheitswert lieferte. Eine Zeichenkette zwingt zum Vergleichen, ein Wahrheitswert führt direkt in eine `if`-Anweisung. Level können damit verlangen, dass die Drohne unterscheidet, worauf sie trifft, statt nur, ob überhaupt.

5.2.2 Lösungsbedingungen

Damit die neuen Handlungen Bedeutung bekommen, wuchs das Levelformat mit. Ein Level kann jetzt zusätzlich zum Portal verlangen, dass Münzen eingesammelt, ein Paket abgeliefert oder eine Kiste auf ein markiertes Feld geschoben wurde. Die Prüfung läuft im Levelmodell, sobald die Drohne das Portal betritt. Ist eine Bedingung offen, meldet die Python-Seite dies über eine `hint`-Nachricht, die benennt, was noch fehlt.

5.3 Aufgabenkurve

Der Levelbestand wuchs von sechs auf dreizehn. Ein Teil davon entstand aus einer Überarbeitung der Einstiegsreihe, bei der die ersten Level neu geordnet und ihre Beschreibungen gestrafft wurden. Die neuen Level greifen die erweiterten Befehle auf, sodass jeder Schritt der Reihe entweder ein Programmierkonzept oder eine Drohnenfähigkeit einführt und möglichst nicht beides gleichzeitig.

Jedes Level trägt seither Schlagworte, die in der Übersicht angezeigt werden und eine grobe Schwierigkeit sowie das behandelte Konzept nennen, etwa Schleifen, Funktionen oder Bedingungen. Diese Vorschau setzt die in Abschnitt 4 beschriebene Passung von Anforderung und Fähigkeit um, weil die Einordnung sichtbar wird, bevor ein Level geöffnet wird. Gesperrte Level bleiben in der Übersicht sichtbar und tragen nur ein Schloss, denn ein sichtbares, aber verschlossenes Ziel wirkt als Anreiz, ein ausgeblendetes nicht. Abb. 2 zeigt die resultierende Übersicht.

5.4 Kennzahlen und Rückmeldung eines Laufs

5.4.1 Kennzahlen

Die Python-Seite führt zwei Zähler. `distance` steigt bei jedem Ortswechsel, `crash_count` bei jeder Kollision. Beide gehen über `get_stats()` an den Hauptthread. Die dritte Kennzahl, die Codezeilenzahl, ermittelt die Oberfläche selbst aus dem eingegebenen Text.

Diese Auswahl folgt der ersten Festlegung aus Abschnitt 4.5. Alle drei Größen beschreiben das Programm, nicht die Person. Wegzahl, Kollisionshäufigkeit und Formulierungskompaktheit sind Eigenschaften des Algorithmus. Die Tippgeschwindigkeit gehört nicht dazu.

5.4.2 Terminal

`print`-Ausgaben liefen ursprünglich nur in die Entwicklerkonsole des Browsers und waren damit für die Zielgruppe praktisch unsichtbar, obwohl `print` das erste Werkzeug für Programmzustände ist. Der Worker leitet die Ausgabe jetzt zusätzlich an den Hauptth-

read weiter, wo sie unter dem Editor in einem Terminalbereich erscheint. Auf demselben Weg laufen auch Hinweistexte der Drohnenbefehle, etwa wenn pickup ins Leere greift.

Der Beitrag zur Gamifizierung ist hier kein Spielelement, sondern eine kürzere Rückmeldeschleife. Ein Fehler mit sichtbarer Ursache führt eher zu einem weiteren Versuch als einer ohne.

5.5 Architektur der Bestenliste

Die Bestenliste ist der einzige Projektteil mit serverseitiger Komponente. Sie läuft als FastAPI-Dienst [6] mit Postgres-Datenbank [8], gestartet zusammen mit dem Frontend über Docker Compose. Der Datenbankzugriff läuft asynchron über SQLAlchemy [7]. Kennwörter werden mit Argon2 gehasht, die Anmeldung liefert ein JWT, und Anmelde- sowie Registrierungsrouen sind ratenbegrenzt.

Drei Tabellen tragen den Dienst. `users` hält die Konten. Ein Index auf der kleingeschriebenen Fassung verhindert Namen, die sich nur in Groß- und Kleinschreibung unterscheiden. `scores` hält je Konto und Level genau die erste Lösung, `attempts` jeden Versuch. Der Vergleich mit anderen beruht damit auf der Erstlösung, während der eigene Verlauf vollständig bleibt. Die Rangfolge selbst richtet sich zuerst nach der Schrittzahl, dann nach der Zeit und zuletzt nach der Codezeilenzahl. Die Zeit läuft als Wanduhr im Hauptthread zwischen Öffnen des Levels und erster Lösung und ist damit unabhängig von Animationsgeschwindigkeit oder Interpreterlaufzeit. Abschnitt 6 zeigt das resultierende Schema und die Oberfläche.

5.5.1 Was gewertet wird

Die Reihenfolge der Kriterien ist die zentrale Entscheidung dieses Strangs. Stünde die Zeit zuerst, bildete die Bestenliste vor allem Tippgeschwindigkeit und Vorkenntnis ab. Die Schrittzahl dagegen ist eine Eigenschaft des gefundenen Wegs und verbessert sich nur durch besseres Verständnis des Levels.

Gewertet wird nur die erste Lösung eines Levels. Die Liste bleibt so ein Vergleich der Lösungsqualität, nicht der aufgewendeten Zeit. Die Tabelle `attempts` hält ergänzend den vollständigen Verlauf, den Angemeldete in der eigenen Fortschrittsanzeige einsehen können.

5.5.2 Betrieb ohne Backend

Die Anwendung bleibt ohne Server benutzbar, weil sie als statischer Export auf GitHub Pages liegt. Fehlt die Umgebungsvariable des API-Clients, liefert jeder Aufruf `null`, und die Bestenliste bleibt ohne Fehler einfach leer. Das Spiel funktioniert dann vollständig, nur ohne Vergleich. Ein Gastmodus im Anmeldedialog macht auch die Anmeldung zur Option statt zur Voraussetzung.

5.6 Level-Editor

5.6.1 Aufbau

Der Editor ist der umfangreichste Einzelbeitrag des Betrachtungszeitraums, über mehrere Durchgänge gewachsen. Er läuft vollständig im Browser und arbeitet auf derselben Datenstruktur wie das Spiel selbst.

Die Oberfläche trennt zwei Modi. Im Bearbeitungsmodus platziert oder löscht ein Klick einen Block auf der aktiven Ebene, im Kameramodus dreht und zoomt derselbe Klick die Ansicht. Die Alt-Taste schaltet vorübergehend auf Kamera um, die X-Taste auf Löschen, denn beide Wechsel sind beim Bauen ständig nötig. Ein Werkzeug legt zusätzlich den Startpunkt der Drohne fest. Alle Ebenen bleiben sichtbar, die aktive hebt ein Gitter hervor, damit die Tiefe beim Bauen erhalten bleibt. Zwei Formulare legen die Ausdehnung in allen drei Achsen sowie Titel, Beschreibung und Schlagworte fest. Der Wechsel per Taste statt per Menüklick hält den Bauprozess nah am Ausprobieren, das auch das Lösen eines Levels prägt.

5.6.2 Vom Entwurf zum Level

Drei Wege führen aus dem Editor heraus. Der Testlauf öffnet das Level in einem Dialog mit vollständiger Spielumgebung, also eigenem Editor, eigener Pyodide-Instanz [4] und eigener Szene, die erst beim Öffnen entsteht und beim Schließen wieder verworfen wird. Der Export schreibt eine YAML-Datei in den Download-Ordner, die Einreichung öffnet ein vorbereitetes GitHub-Issue mit bereits eingetragenen YAML. Editor und Spiel teilen sich dafür dieselbe Datenstruktur ohne Umweg über das Dateisystem.

Lösungsbedingungen mussten Autoren ursprünglich von Hand eintragen. Der Editor leitet sie inzwischen aus dem gebauten Level ab. Wer drei Münzen platziert, baut ein Level mit drei einzusammelnden Münzen. Geometrie und Bedingung laufen so nicht mehr auseinander, und der Bauvorgang wird kürzer.

5.7 Reibungsabbau

Der sechste Strang bündelt Arbeiten, die selbst keine Spielelemente sind, aber Voraussetzung dafür, dass die anderen fünf wirken.

Die Darstellung auf schmalen Bildschirmen wurde überarbeitet. Betroffen waren der Ebenenwähler des Editors, ein nie gebundenes Mausradereignis und eine überfüllte Kopfzeile. Da ein erheblicher Teil der Zugriffe von Mobilgeräten kommt, entscheidet diese Ebene, ob die erste Aufgabe überhaupt erreicht wird.

Seit Juli fängt eine eigene Fehlerseite mit animierter Drohne ungültige Level-Adressen ab, statt eine Fehlermeldung des Servers zu zeigen. Das ist kein Spielelement, verhindert aber, dass eine falsche Adresse wie ein Defekt wirkt.

Schließlich entstand eine automatisierte Testabdeckung. Die Spieltests täuschen das Brückenmodul zur JavaScript-Seite vor, sodass der unveränderte Quelltext unter CPython läuft. Die Dienst-Tests laufen im selben Prozess je Test gegen eine wegwerfbare SQLite-Datenbank statt gegen Postgres. So ist die im Browser schwer prüfbare Spiellogik in einer gewöhnlichen Testumgebung abgesichert.

6 Ergebnisse

Die in Abschnitt 5 beschriebenen Arbeitsstränge haben eine zuvor nicht vorhandene, geschlossene Spielschleife hervorgebracht. Jede Auswertung führt jetzt auf eine mögliche nächste Handlung, statt den Ablauf mit dem Ziel enden zu lassen. Tab. 1 fasst den quantitativen Zuwachs zwischen v1.0.0 und dem Ende des Betrachtungszeitraums zusammen.

Kennzahl	v1.0.0	28.09.2026
Spielbare Level	6	13
Öffentliche Drohnen-Methoden	10	14
Blocktypen	9	14
Testfunktionen	0	118

Tab. 1: Wachstum der zentralen Kennzahlen zwischen dem Ausgangsstand v1.0.0 und dem Ende des Betrachtungszeitraums.

6.1 Neue Bausteine und Levelübersicht

Neu in der Blockregistrierung sind Münze, Kiste, Ablagefläche und Zielmarkierung. Abb. 1 zeigt ein Level, das drei davon gleichzeitig nutzt. Abb. 2 zeigt die in Abschnitt 5 beschriebene Levelübersicht mit Schwierigkeits- und Konzept-Schlagworten.

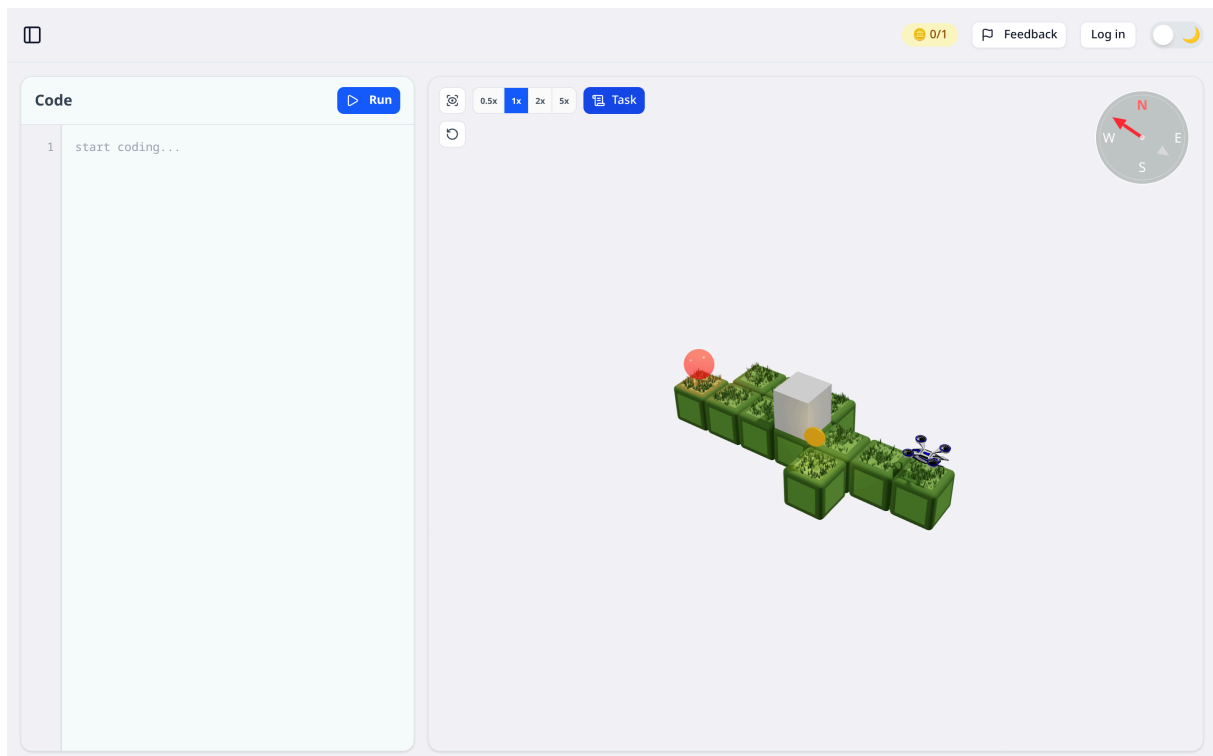


Abb. 1: Level *Push 'n Collect* vor dem ersten Lauf. Die Kiste versperrt den Weg, die Münze liegt auf einem Seitenpfad.

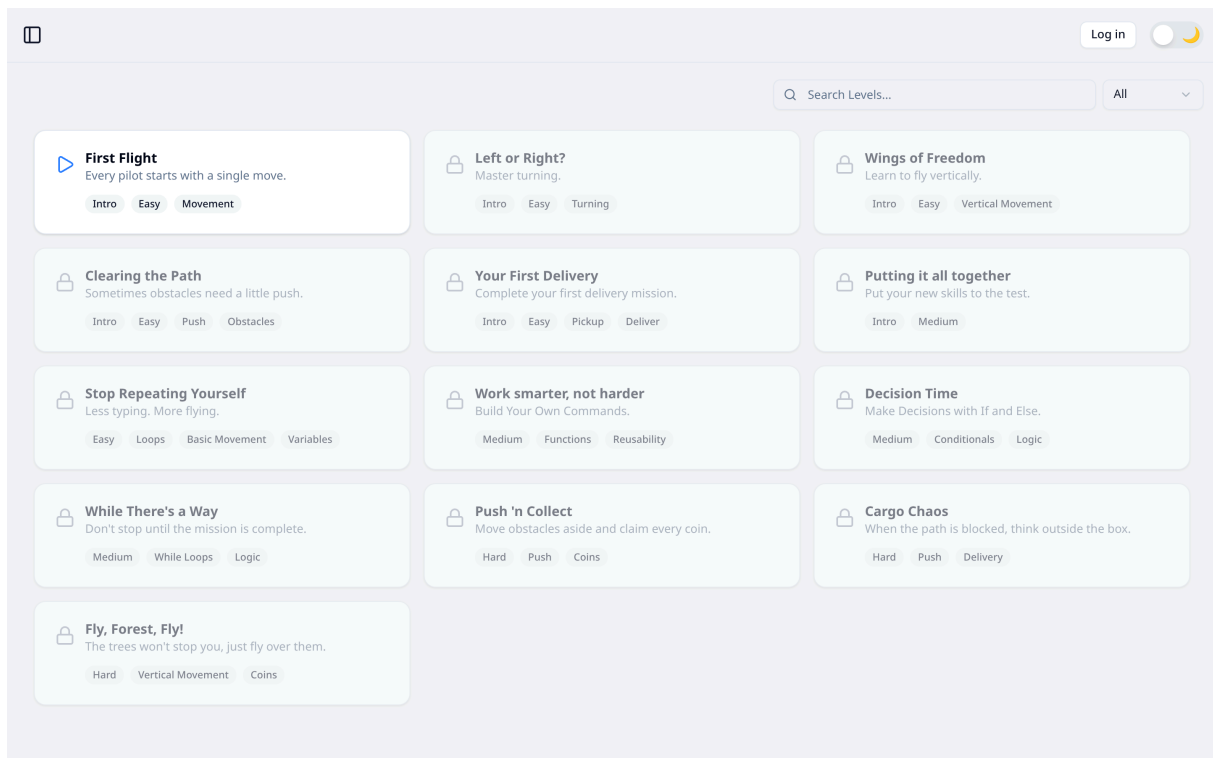


Abb. 2: Levelübersicht mit dreizehn Einträgen. Schlagworte kündigen Schwierigkeit und Konzept an, gesperrte Level tragen ein Schloss.

6.2 Rückmeldung und Bestenliste

Beim Erreichen des Ziels erscheint eine Anzeige mit Zeit, den drei Kennzahlen und einem knappen, an der Absturzzahl bemessenen Kommentar (Abb. 3). Abb. 4 zeigt das resultierende Datenbankschema. Jedes Konto in `users` kann beliebig viele Zeilen in `scores` und `attempts` besitzen, `scores` erzwingt zusätzlich Eindeutigkeit je Konto und Level. Abb. 5 zeigt die daraus gespeiste Bestenliste, deren Sortierung zuerst der Schrittzahl folgt.

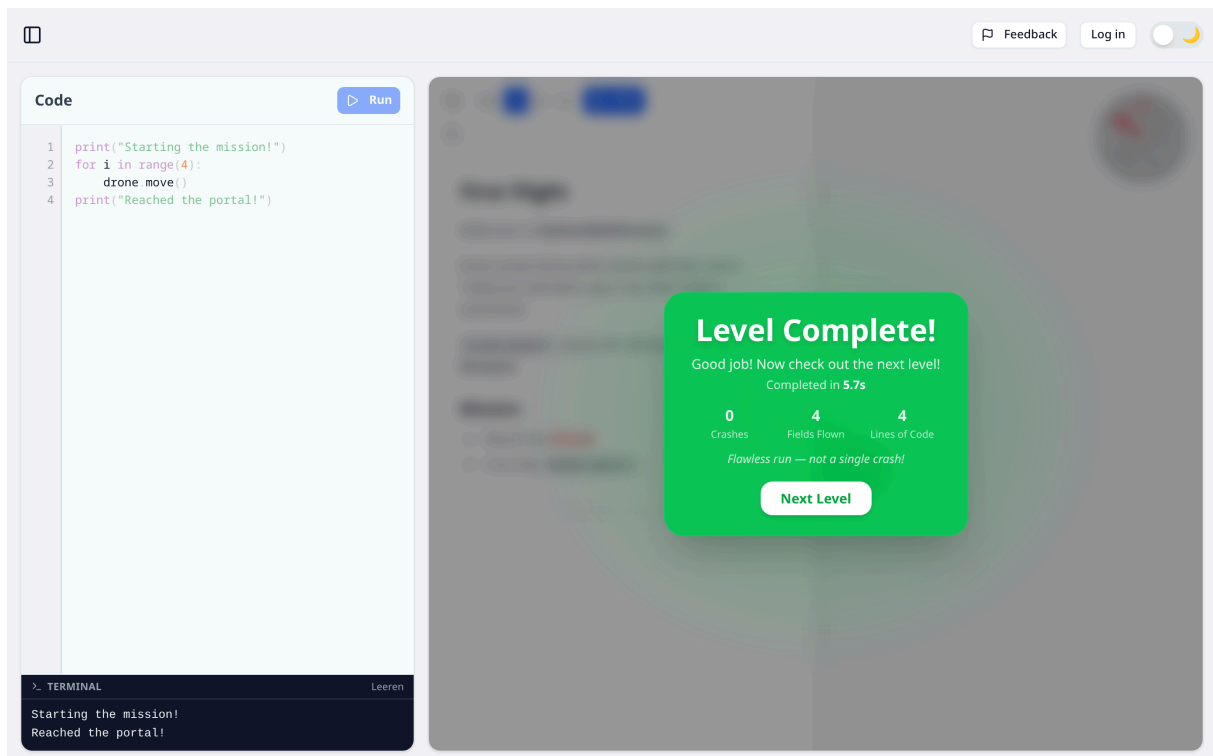


Abb. 3: Die Abschlussanzeige nach einem gelösten Level mit Zeit, Abstürzen, geflogenen Feldern und Codezeilen. Unten links zeigt das Terminal die Ausgaben der beiden `print`-Aufrufe.

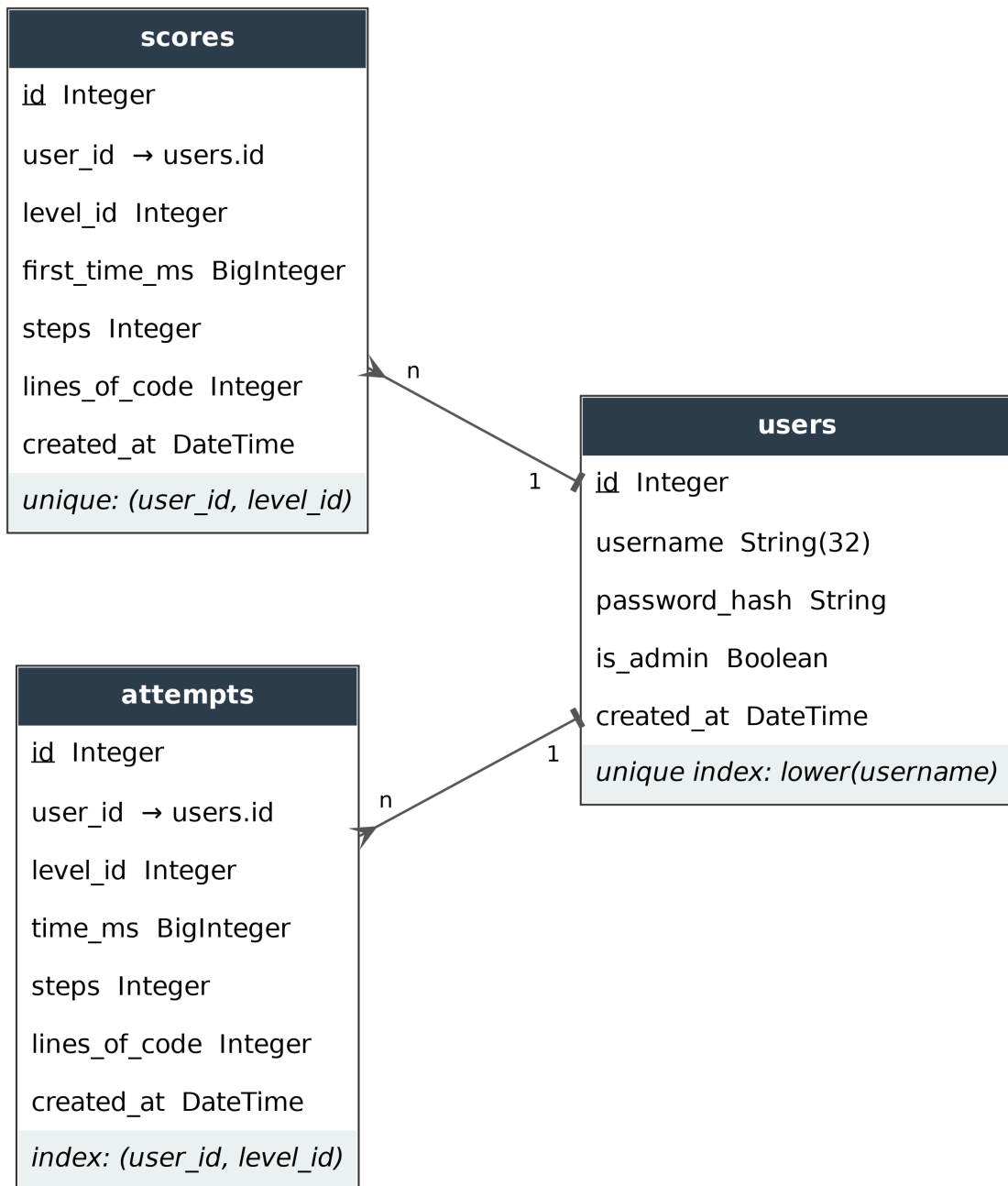


Abb. 4: Datenbankschema der Bestenliste.

RANK	PLAYER	STEPS	TIME	LINES OF CODE	WHEN
1	Niclas	9	48.2s	11	26 Jul 2026, 21:19
2	Stefan	9	1:11.4s	14	26 Jul 2026, 21:19
3	Jonas	11	1:03.9s	9	26 Jul 2026, 21:19
4	Mira	12	2:35.2s	21	26 Jul 2026, 21:19
5	Tom	16	3:28.7s	26	26 Jul 2026, 21:19

Abb. 5: Die Bestenliste einer laufenden Instanz. Die Reihenfolge folgt zuerst der Schrittzahl.

6.3 Level-Editor und mobile Darstellung

Abb. 6 zeigt den Level-Editor mit einem geladenen Level, links Modus, Werkzeuge, Blockpalette und Größenangaben, oben die Aktionen Export, Einreichung, Testlauf und Speichern. Abb. 7 zeigt die Levelansicht auf einem schmalen Bildschirm. Szene und Editor stehen dort untereinander statt nebeneinander.

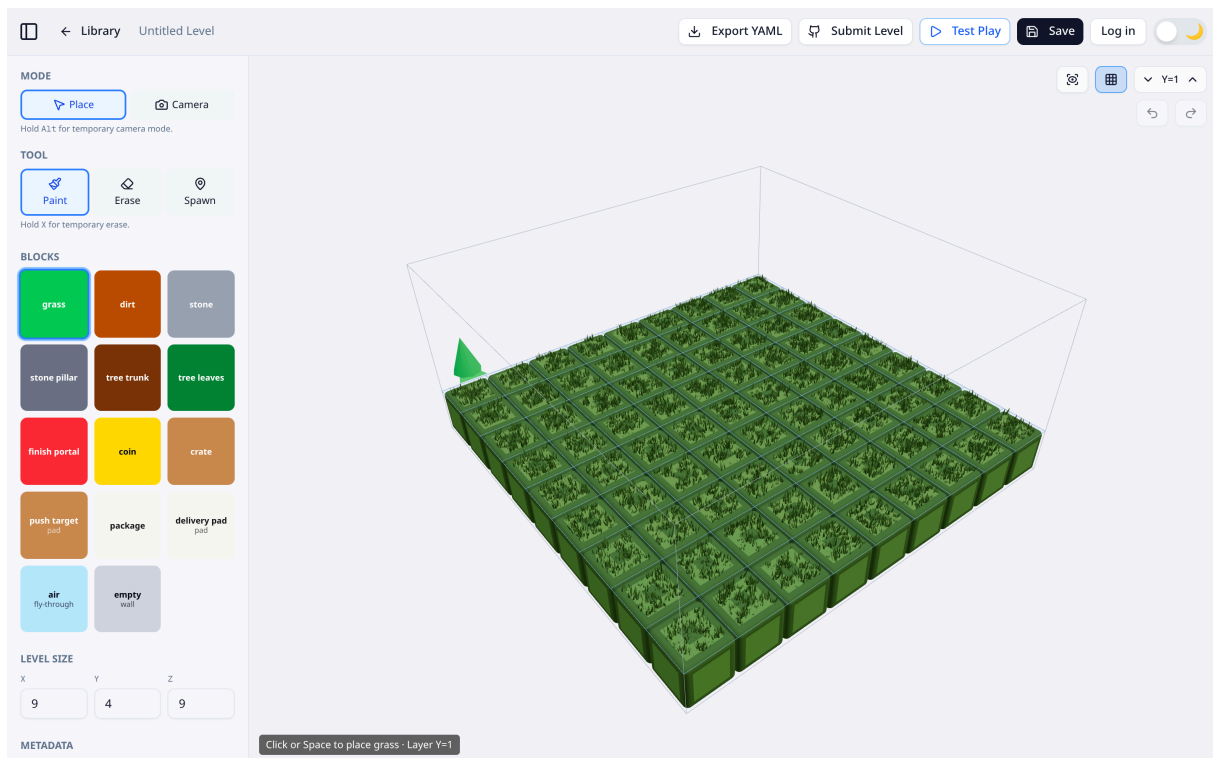


Abb. 6: Der Level-Editor mit einem geladenen Level.

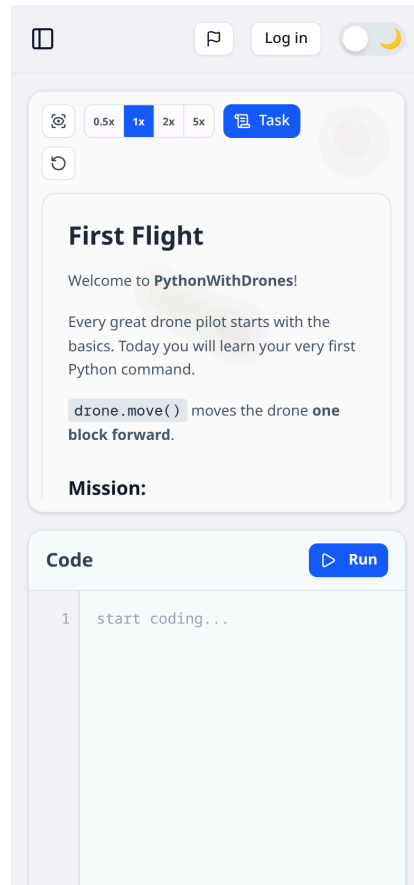


Abb. 7: Levelansicht auf schmalem Bildschirm.

7 Diskussion

7.1 Zuordnung zu den Grundbedürfnissen

Kennzahlen, Abschlussanzeige, Versuchsverlauf, gestufte Levelreihe, Terminal und Hinweise bei offenen Lösungsbedingungen machen Fortschritt abgestuft statt binär sichtbar und geben Fehlschlägen eine benennbare Ursache. Das bedient die Kompetenz. Der Level-Editor adressiert Autonomie, die Punkte und Bestenliste nicht bedienen. Er verschiebt die Rolle vom Lösen vorgegebener Aufgaben zum Stellen eigener, ohne dafür ein Konto zu verlangen. Das Bauen eines lösbaren Levels erfordert zudem, die Simulationsregeln vollständig zu verstehen, und der Editor senkt zugleich den Aufwand für neue Level von handgeschriebenem YAML auf ein Formular. Sozial eingebunden wird, wer über Bestenliste, Kontoname oder eigene Level in Bezug zu anderen tritt. Der vorher rein lokale Fortschritt bekommt so einen Rahmen außerhalb des eigenen Browsers.

Am weitesten ausgebaut ist der Kompetenzstrang, weil er auf jeden Lauf unmittelbar zurückwirkt. Die soziale Ebene ist mit Bestenliste und eingereichten Leveln die jüngste und bietet den größten Spielraum für Weiterarbeit.

7.2 Einschränkungen und Reichweite der Aussagen

Die Wirkungsaussagen dieser Arbeit sind Entwurfsargumente aus dem gebauten System und den vorangestellten Festlegungen, keine empirischen Befunde. Eine Nutzerstudie war im Zeitrahmen nicht vorgesehen.

Drei weitere Einschränkungen bestehen.

- Fehlende Funktionalität: Abzeichen als dritter Gamifizierungs-Baustein sind zurückgestellt, eine serverseitige Plausibilitätsprüfung der Kennzahlen existiert nicht.
- Verwendete Mittel: Level verlassen den Editor nur per YAML-Export oder GitHub-Issue, ohne GitHub-Konto lässt sich keines veröffentlichen.
- Qualität: Die Bestenliste vertraut den Client-Kennzahlen vollständig, in einem offenen Wettbewerb ließe sich das ausnutzen.

7.3 Bewusst getroffene Festlegungen

Mehrere Entwurfsentscheidungen hatten Alternativen, die aus guten Gründen nicht gewählt wurden.

Kennzahlen werden im Browser gezählt und unverändert übernommen. Das hält die Ausführung clientseitig und die Bestenliste ohne zusätzliche Rechenlast. Für eine Lehrveranstaltung reicht dieses Vertrauensmodell aus. In einem offenen Wettbewerb müsste der Dienst die Lösung selbst nachrechnen.

Die Bestenliste bringt eine serverseitige Komponente in ein zuvor serverloses Projekt. Eine Umgebungsvariable schaltet sie bei Bedarf still ab, statt sie zur Voraussetzung zu machen. Dieselbe Auslieferung bleibt so mit und ohne Dienst vollständig spielbar. Der Gastmodus hält zusätzlich die zweite Festlegung aus Abschnitt 4.5 ein, denn kein Lerninhalt hängt an einer Belohnung.

Im Editor gebaute Level liegen im `localStorage` und verlassen ihn nur über YAML-Export oder GitHub-Issue. Für die Übergabe an das Projektteam genügt das, ohne den Editor an eine Kontopflicht zu binden.

7.4 Wertungslogik und Motivation

Eine Belohnung, die auf etwas anderes zeigt als das Lernziel, kann die ursprüngliche Motivation überlagern [12]. Das war der maßgebliche Prüfpunkt beim Entwurf der Wertung. Zwei Entscheidungen begrenzen dieses Risiko. Die Rangfolge nach Schritten belohnt Levelverständnis statt bloßen Fleiß, und die Beschränkung auf die erste Lösung verhindert, dass Wiederholung allein die Liste verbessert. Beide zusammen richten die Belohnung auf dieselbe Fähigkeit, die die Anwendung vermitteln soll.

8 Schlussfolgerung

8.1 Zusammenfassung

Zu Modulbeginn führte die Anwendung Python im Browser aus und animierte eine Drohne, doch der Ablauf endete mit dem Ziel. Jetzt besteht eine geschlossene Schleife. Ein Level stellt eine Aufgabe, eine Auswertung bewertet die Lösung nach Schritten, Abstürzen und Codezeilen, eine Bestenliste stellt sie neben andere, und ein Editor öffnet den Weg zur nächsten Aufgabe.

Die Kennzahlen zeigen den Umfang. Drohnen-Methoden wuchsen von zehn auf vierzehn, Blocktypen von neun auf vierzehn, Level von sechs auf dreizehn. Hinzu kamen ein optionaler FastAPI-Dienst mit Konten, Rangfolge und Versuchsverlauf, ein dreidimensionaler Level-Editor mit Testlauf und GitHub-Einreichung sowie 118 Testfunktionen, die Spiellogik und Bestenlisten-Dienst außerhalb des Browsers absichern.

8.2 Nächste Schritte

Eine Nutzerstudie mit und ohne aktivierte Bestenliste könnte die hier begründeten Wirkungen empirisch prüfen. Eine serverseitige Plausibilitätskontrolle, etwa eine untere Schranke der Schrittzahl, würde die eingereichten Kennzahlen absichern. Abzeichen für das erstmalige Lösen eines Levels mit Schleife, Funktion oder Bedingung könnten den zurückgestellten dritten Gamifizierungs-Baustein besetzen. Eine Ablage eingereichter Level im Backend würde sie ohne GitHub-Konto veröffentlichbar machen und ließe sich zu einer rotierenden Auswahl kuratieren, die die soziale Ebene über den Rangvergleich hinaus erweitert.

Literatur

- [1] N. Falke, S. Reitemeyer, und J. Pottmeier, „Python with Drones“, Ausarbeitung im Modul Software Design, 2026.
- [2] „PythonWithDrones, Quelltext-Repository“. [Online]. Verfügbar unter: <https://github.com/pottmeier/PythonWithDrones>
- [3] „Next.js Documentation“. [Online]. Verfügbar unter: <https://nextjs.org/docs>
- [4] „Pyodide Documentation“. [Online]. Verfügbar unter: <https://pyodide.org/>
- [5] „React Three Fiber Documentation“. [Online]. Verfügbar unter: <https://r3f.docs.pmnd.rs/>
- [6] „FastAPI Documentation“. [Online]. Verfügbar unter: <https://fastapi.tiangolo.com/>
- [7] „SQLAlchemy Documentation“. [Online]. Verfügbar unter: <https://docs.sqlalchemy.org/>
- [8] „PostgreSQL Documentation“. [Online]. Verfügbar unter: <https://www.postgresql.org/docs/>
- [9] S. Deterding, D. Dixon, R. Khaled, und L. Nacke, „From Game Design Elements to Gamefulness: Defining "Gamification"“, in *Proceedings of the 15th International Academic MindTrek Conference: Envisioning Future Media Environments*, ACM, 2011, S. 9–15.
- [10] K. Werbach und D. Hunter, *For the Win: How Game Thinking Can Revolutionize Your Business*. Wharton Digital Press, 2012.
- [11] R. Hunicke, M. LeBlanc, und R. Zubek, „MDA: A Formal Approach to Game Design and Game Research“, in *Proceedings of the AAAI Workshop on Challenges in Game AI*, 2004.
- [12] E. L. Deci und R. M. Ryan, „The "What" and "Why" of Goal Pursuits: Human Needs and the Self-Determination of Behavior“, *Psychological Inquiry*, Bd. 11, Nr. 4, S. 227–268, 2000.
- [13] M. Csikszentmihalyi, *Flow: The Psychology of Optimal Experience*. Harper, Row, 1990.